Corso di Fondamenti di Informatica 2
CdL Ingegneria Informatica
Ing. Franco Zambonelli

ARCHITETTURA DEGLI ELABORATORI: ESERCIZI ASSEMBLER (Completi + Traduzioni dal C)

Lucidi Realizzati in Collaborazione con:

Prof. Letizia Leonardi
Università di Modena

Prof. Antonio Corradi
Università di Bologna

ESEMPIO

SCRITTURA DEL SEGUENTE PROGRAMMA:

```
#define N 10
int s, v[]= {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

main()
{
        int i;
        s=0;
        for (i=0; i<=N; i++)
                s = s + v[i];
};
```

ESEMPIO: Programma che somma i numeri da 0 a 10
Soluzione A ===> solo uso di registri

```
CODICE segment 'CODE'
assume   cs:CODICE, ss:ST1
_mainproc      far
        push    ds        ; preparazione del
        mov     ax, 0     ; ritorno al sistema operativo
        push    ax
        mov     di, 0 ;   DI rappresenta la variabile s
        mov     si, 0 ;   SI rappresenta la variabile i
verifica:
        cmp     si,10
        jg      short fine
ciclo:
        mov     ax, di
        add     ax, si
        mov     di, ax
        inc     si
        jmp     short verifica
fine:
        ret
_mainendp
CODICE  ends

ST1     segment STACK 'STACK'
          db   256 dup('stack   ') ; byte 8 * 256
ST1     ends
        end _main ;  entry point  non necessario
```


Soluzione B ===> uso di registri e variabili

```
CODICE segment 'CODE'
assume   cs:CODICE, ds: DATI, ss:ST1
_main   procfar
        push    ds
        mov     ax, 0
        push    ax
        mov     ax, DATI      ;
        mov     ds, ax        ;  DS <- DATI
        mov     s, 0
        mov     si, 0       ; SI contiene la variabile i
verifica:
        cmp     si, N
        jg      short fine
ciclo:  mov     ax, s
        add     ax, si
        mov     s, ax
        inc     si
        jmp     short verifica
fine:ret
_mainendp
CODICE          ends
DATI  segment 'DATA'
N       dw  10
s       dw  ?
DATI ends
ST1     segment STACK 'STACK'
          db   256 dup('statck')
ST1     ends
        end
```

Soluzione C ===> uso di **registri e variabili**

```
CODICE segment 'CODE'
assume   cs:CODICE, ds: DATI, ss:ST1
_mainproc       far
          push    ds
          mov     ax, 0
          push    ax
          mov     ax, DATI
          mov     ds, ax
          mov     s, 0
          mov     si, 0
verifica: cmp     si, word ptr N
          jg      short fine
ciclo:    mov     ax, word ptr s
          add     ax, si
          mov     word ptr s, ax
          inc     si
          jmp     short verifica
fine:ret
_mainendp
CODICE          ends
DATI segment 'DATA'
N         db  10
s         db  ?
DATI ends
ST1   segment STACK 'STACK'
          db   256 dup('statck')
ST1   ends
          end
```


Soluzione D ===> ***indirizzamento INDIRETTO tramite registro BASE***

```
CODICE segment 'CODE'
assume   cs:CODICE, ds: DATI, ss:ST1
_mainproc       far
          push    ds
          mov     ax, 0
          push    ax
          mov     ax, DATI
          mov     ds, ax
          mov     di, 0
          mov     si, 0
          mov     bx, P
verifica:
          cmp     si, [bx]
          jg      short fine
ciclo:    mov     ax, di
          add     ax, si
          mov     di, ax
          inc     si
          jmp     short verifica
fine:ret
_mainendp
CODICE          ends
DATI segment 'DATA'
N         dw  10
P       dwN
DATI ends
ST1 ...
          end
```

Soluzione E ===> indirizzamento INDIRETTO
tramite registro INDICE dati memorizzati in un vettore

```
CODICE segment 'CODE'
assume   cs: CODICE, ds: DATI, SS: ST1
_mainproc       far
        push    ds
        mov     ax, 0
        push    ax
        mov     ax, DATI
        mov     ds, ax
        mov     s,0
        mov     si, 0
verifica:
        cmp     si,10
        jg      short fine
        mov     di, si
        shl     di, 1       ; moltiplicazione per DUE
        mov     ax, s
        add     ax, [v+di]          ; v[di]
        mov     s,ax
        inc     si
        jmp     short verifica
fine:ret
_mainendp
CODICE ends
```



```
DATI segment 'DATA'
s     dw  ?
v     dw      0     ;  usare la REPT per ottenere
      dw      1     ;  la stessa definizione
      dw      2
      dw      3
      dw      4
      dw      5
      dw      6
      dw      7
      dw      8
      dw      9
      dw      10
DATI ends

ST1   segment STACK 'STACK'
      db 256 dup ('stack   ')
ST1   ends

      end
```

ESERCIZIO:
Procedura che somma un range di numeri interi
da inizio a fine

somma = inizio + (inizio+1) ... (fine-1) + fine

Versione C:

```
int somma (int inizio, int fine)
{  int s, i;
      s = 0;
      for (i = inizio; i <= fine; i++)       s = s + i;
      return s;
}
```

Versione ASSEMBLER:

```
_TEXT    segment byte public 'CODE'
assume   cs:_TEXT, ds:_TEXT, ss: st1
;      int somma (int inizio, int fine)
;
_somma      proc   near ; inizio procedura somma
      push     bp              ; salvataggio BP chiamante
      mov      bp,sp           ; posizionamento BP
      push     si              ; salvataggio registri usati
      push     di

;  {    int s, i;         ; questa versione usa
;       s = 0;            ; DI per s e SI per i
; cioe' la traslazione utilizza i REGISTRI della macchina
;  ===>   OTTIMIZZAZIONE
      xor      di,di    ; azzeramento DI (cioè s)
;
```



```
;       for (i = inizio; i <= fine; i++)
      mov      si,word ptr [bp+4] ; SI <== inizio
      jmp short verifica                  ; verifica la fine del for
ciclo:
;
;           s = s + i;
      mov      ax,di      ; AX <== DI (somma corrente)
      add      ax,si      ; AX <== DI + SI (operando corrente)
      mov      di,ax      ; In DI la somma corrente
      inc   si            ; incremento di SI (cioe' i)

verifica:
      cmp      si,word ptr [bp+6]; verifica la fine del for
      jle      short ciclo

;
;           return s;
      mov      ax,di      ; in AX c'e' la somma risultante
;
;     }
      pop      di         ; ripristino registri usati
      pop      si
      pop      bp         ; ripristino BP
      ret                 ; si ritorna
_somma  endp
```

```
_mainproc       far
        push    ds          ; memorizza nello stack l'indirizzo
        mov     ax, 0       ; cui saltare al RET
        push    ax          ; (a ds:0000 la chiamata a int 20H)

;       S = somma(0, 10);

        mov ax,10
; SECONDO parametro sullo stack (fine=10)
        push    ax
        xor     ax,ax
; PRIMO parametro sullo stack (inizio=0)
        push    ax
        call    near ptr _somma  ; chiamata
        pop     cx          ; estrazione dei parametri
        pop     cx          ; dallo stack

;       in AX c'e' il risultato della procedura
        ret                 ; terminazione
_main   endp
_TEXT   ends
st1     SEGMENT stack 'STACK'
        db      256 dup('stack     ')
st1     ENDS
    end   _main
```


ESERCIZIO:
Nell'esercizio precedente la traslazione veniva fatta utilizzando i REGISTRI : questa volta **NON** utilizziamo i registri

le variabili locali sono inserite nel record di attivazione della singola invocazione di somma
; s == **BP *(registro record di attivazione)*** - 2

Procedura ASSEMBLER somma:

```
_TEXT   segment byte public 'CODE'
assume  cs:_TEXT, ds:_TEXT, ss: st1
_somma  proc  near
;int somma (int inizio, int fine)
        push    bp
        mov     bp,sp
        sub     sp,4

; si riserva spazio per le variabili locali s ed i
        mov     word ptr [bp-2],0       ; s <== 0

;       for (i = inizio; i <= fine; i++)
        mov     ax,word ptr [bp+4]      ; inizio
        mov     word ptr [bp-4],ax      ; i <== inizio
        jmp     short  verifica
```

```
ciclo:
;       s = s + i;
        mov     ax,word ptr [bp-2]          ; s corrente
        add     ax,word ptr [bp-4]          ; AX <== s + i
        mov     word ptr [bp-2],ax          ; nuovo s
        inc     word ptr [bp-4]     ; i++

verifica:
        mov     ax,word ptr [bp-4]      ; i corrente
        cmp     ax,word ptr [bp+6]      ; i <= fine ?
        jle     short ciclo

;       return s;
        mov     ax,word ptr [bp-2] ; AX <== s corrente
        mov     sp,bp
        pop     bp
        ret
_somma  endp
```


ESERCIZIO:
Procedura **RICORSIVA** che somma un range di numeri interi

Versione C:

```
int somma (int inizio, int fine)
{       if (inizio < fine)
                return (inizio + somma (inizio + 1, fine));
        else if (inizio==fine)  return inizio;
                else return 0
;
}
```

Versione ASSEMBLER:

```
_TEXT   segment byte public 'CODE'
assume  cs:_TEXT, ds:_TEXT, ss: st1
;
;       int somma (int inizio, int fine)
;
_somma      proc      near
        push    bp          ; salvataggio BP chiamante
        mov     bp,sp       ; posizionamento BP
        push    si          ; salvataggio registro
        mov     si,word ptr [bp+4] ; SI <== inizio
;
;       {       if (inizio < fine)
;
        cmp     si,word ptr [bp+6]; inizio  >= fine ?
        jge     short fine
```

```
;
;       return (inizio + somma (inizio+1, fine));
;
        push    word ptr [bp+6]
; secondo parametro (fine) sullo stack
        mov     ax,si     ; AX <== inizio
        inc     ax     ; inizio +1
        push    ax        ; 1o parametro (inizio+1) sullo stack
        call    near ptr _somma  ; chiamata RICORSIVA
        pop     cx        ; togliamo i parametri
        pop     cx        ; dallo stack
        add     ax, si    ; AX <== inizio + risultato (AX)
        jmp     last
fine:
;       else if (inizio==fine)  return inizio;
        cmp     si,word ptr [bp+6] ; inizio  == fine ?
        je      short fineultima
;           else return 0;
        mov     ax, 0     ; inizio > fine
        jmp     last
fineultima:
        mov     ax, si    ; inizio == fine
last:
        pop     si      ; ripristino registri
        pop     bp
        ret               ; ritorno
_somma      endp
```


ESEMPIO:

PROGRAMMA IN C CHE USA UNA FUNZIONE SCRITTA IN ASSEMBLER (MODELLO SMALL)

Programma C:

```
extern int f(char c, int b); /* funzione Assembler */

void main()
{
      int A, B;
      char C;

      printf("Scrivi un carattere e un numero intero\n");
      scanf("%c %d", &C, &B);
      A = f(C, B);
      printf("Il valore risultante e' %d\n", A);
}
```

Traduzione della funzione

```
int f(char c, int b)
{
      if (c == 'a') return 0;
      else return b;
}
```

Programma Assembler:

```
_TEXT   segment byte public 'CODE'
assume  cs:_TEXT
public  _f
;
;   int f(char c, int b)
_f      proc    near
        push    bp
        mov     bp,sp
;   {
;   if (c == 'a') return 0;
;
        cmp     byte ptr [bp+4], 'a'
        jne     short else
        xor     ax,ax
        jmp     short fine
else:
;   else return b;
        movax,word ptr [bp+6]
fine:
;   }
        pop     bp
        ret
_f      endp
_TEXT   ends
        end
```


ESEMPIO:

Programma C che invoca una procedura ASSEMBLER

Programma C:

```
extern int f(int a, int b); /* PROCEDURA ASSEMBLER */

void main()
{   int A, B, C;

    printf("Scrivi due valori interi\n");
    scanf("%d %d", &A, &B);
    C = f(A, B);    /* chiamata ad una procedura assembler */
    printf("Il valore risultante e' %d\n", C);
}
```

Traduzione della funzione

```
int f(char a, int b)
{
    if ( a== b) return (a);
    else if (a > b) return ( f (a , b + 1));
        else return ( f (a + 1, b));
}
```

Programma ASSEMBLER:

```
_TEXT segment byte public 'CODE'
assume  cs:_TEXT
public        _f

_f      proc      near
          push      bp
          mov       bp,sp
          push      si
          push      di
          mov       si,word ptr [bp+4]
          mov       di,word ptr [bp+6]
;     {  if (a == b) return (a);
          cmp       si,di
          jne       short ricorri
          mov       ax,si
          jmp       short fine
ricorri:
;         else if (a > b) return (f (a, b + 1));
          cmp       si,di
          jle       short altern
          mov       ax,di
          inc       ax
          push      ax                ; inserimento 2 parametro
          push      si                ; inserimento 1 parametro
          call      near ptr _f       ; chiamata ricorsiva
          pop       cx
          pop       cx
          jmp       short fine
```



```
;       else return ( f (a + 1, b));
altern:
          push      di                ; inserimento 2 parametro
          mov       ax,si
          inc       ax
          push      ax                ; inserimento 1 parametro
          call      near ptr _f       ; chiamata ricorsiva
          pop       cx
          pop       cx
fine:
  ;     }
          pop       di
          pop       si
          pop       bp
          ret
_f      endp
_TEXT   ends
        end
```

ESEMPIO:

Programma C che chiama una procedura ASSEMBLER, che a sua volta chiama una funzione C

Programma C:

```
extern int f(int a, int b);

int prova(int a)
{return (a+1);}

void main()
{      int A, B, C;

   printf("Scrivi due valori interi\n");
   scanf("%d %d", &A, &B);
   C = f(A, B);
   /* chiamata ad una procedura ASSEMBLER */
   printf("Il valore risultante e' %d\n", C);
}
```

Traduzione della funzione:

```
int f(int a, int b);
{      if (a= = b) return prova(a);
       else  if (a > b)  return prova(f(a, b+1));
             else         return prova(f(a+1, b));
}
```


Programma ASSEMBLER:

```
_TEXT   segment byte public 'CODE'
assume  cs:_TEXT
public     _f
extrn      _prova

_f       proc       near
              push       bp
              mov        bp,sp
              push       si
              push       di
              mov        si,word ptr [bp+4]
              mov        di,word ptr [bp+6]
              cmp        si,di
              jne        short ricorri
              mov        ax,si
              jmp        short fine
ricorri:
              cmp        si,di
              jle        short altern
              mov        ax,di
              inc        ax
              push       ax
              push       si
              call       near ptr _f
              pop        cx
              pop        cx
              jmp        short fine
```

```
altern:
        push    di
        mov     ax,si
        inc     ax
        push    ax
        call    near ptr _f
        pop     cx
        pop     cx
fine:
        push    ax; parametro della funzione C
        call    near ptr _prova
; chiamata alla funzione C
; AX contiene in valore risultante della funzione _prova
; che è il parametro di ritorno della funzione _f
        pop     cx          ; estrazione parametro

        pop     di
        pop     si
        pop     bp
        ret
_f      endp
_TEXT   ends

        end
```


ESEMPIO:

Programma C che chiama una procedura ASSEMBLER, che a sua volta chiama una funzione della libreria C

Programma C:

```
extern void f(int a, int b);

void main()
{
int A, B, C;
printf("Scrivi due valori interi\n");
scanf("%d %d", &A, &B);
C = f(A, B);
}
```

Traduzione della funzione:

```
void f(int a, int b);
{     if (a= = b) printf(“Valore risultante %d\n”, a);
      else  if (a > b)  prova(f(a, b+1));
            else        prova(f(a+1, b));
}
```

Programma ASSEMBLER:

```
_TEXT   segment byte public 'CODE'
assume  cs:_TEXT, ds: DGROUP
public     _f
extrn      _printf

_f      proc       near
      push      bp
      mov       bp,sp
      push      si
      push      di

            mov       si,word ptr [bp+4]
            mov       di,word ptr [bp+6]
            cmp       si,di
            jne       short ricorri

; invocazione della printf (chiamata vicina)
            mov       ax,si
            push      ax    ; secondo parametro
            lea       bx, string
            push      bx    ; primo parametro (stringa formato)
            call      near ptr _printf ; chiamata alla funzione C
            pop       cx    ; estrazione parametri
            pop       cx

            jmp       short fine
ricorri:
            cmp       si,di
            jle       short altern
```



```
            mov       ax,di
            inc       ax
            push      ax
            push      si
            call      near ptr _f
            pop       cx
            pop       cx
            jmp       short fine
altern:
            push      di
            mov       ax,si
            inc       ax
            push      ax
            call      near ptr _f
            pop       cx
            pop       cx
fine:       pop       di
            pop       si
            pop       bp
            ret
_f      endp
_TEXT   ends
DGROUP group _DATA
_DATA segment word public 'DATA'
string      db    'Valore risultante %d',0dh, 0ah, 0
; stringa con ritorno DOS (<CR><LF>) e fine stringa
_DATA ends
end
```

ESEMPIO:

Programma C che chiama una procedura ASSEMBLER
USO DI TUTTI I MODELLI DI MEMORIA!

Programma C (file main.c):

```
#include <string.h>

extern int lung(char *s);
/* procedura ASSEMBLER */

char *string = "Ciao ciao!";
/* dato globale inizializzato */

int r;
/* dato globale NON inizializzato */

void main()
{
      r = lung(string);
      printf ("Lunghezza stringa  %d\n", r);
      /* verifica tramite l'uso della funzione di libreria */
      printf ("Lunghezza stringa %d\n", strlen(string));
}
```

Versione C della procedura lung (file lungX.[c | asm]):

```
int lung(char *s)
{     int i=0;
      while (*s++) i++;
      return i;
}
```


MODELLI TINY e SMALL:

si usa lo stesso codice

Programma ASSEMBLER (file lungts.asm):

```
_TEXT   segment byte public 'CODE'
public  _lung  ;  int lung(char *s)
assume  cs:_TEXT
_lung proc      near
        push    bp
        mov     bp,sp
        push    si          ; salvataggio di SI
;   {
;   int i=0;
        xor     si,si       ; si usa SI per la variabile i
;   while (*s++) i++;
ciclo:  mov     bx,word ptr [bp+4]
        inc     word ptr [bp+4]
        cmp     byte ptr [bx],0
        je      short fine
        inc     si
        jmp     ciclo

;   return i;
fine:mov    ax,si
;   }
        pop     si
        pop     bp
        ret
_lung   endp
_TEXT   ends
end
```

MODELLO TINY

produzione eseguibile ===> **maint.exe**

tcc -mt maint.c lungts.asm

Assembler corrispondente a mainl.c ===>

```
DGROUP    group _DATA, _BSS
assume    cs: _TEXT, ds: DGROUP
_TEXT     segment byte public 'CODE'
;     void main()
_mainproc       near
          push      bp
          mov       bp,sp
          ...
          pop       bp
          ret
_mainendp
_TEXT     ends

_BSS  segment word public 'BSS'
_r    db   2 dup (?) ; dati NON inizializzati
_BSS  ends

          end
```



```
_DATA segment word public 'DATA'
; dati inizializzati
_string   db        'Ciao ciao!'
          db        0
          db        'Lunghezza stringa %d'
          db        0dh, 0ah
          db        0
          db        'Lunghezza stringa %d'
          db        0dh, 0ah
          db        0
_DATA     ends
```

OSSERVAZIONI:

a) A TEMPO DI ESECUZIONE unico segmento per codice, dati e stack

===> DS = SS = CS

DGROUP (_TEXT, _DATA, _BSS, _STACK)

il modello **cliente/servitore** usa sempre lo stesso segmento:

chiamate vicine (indirizzi intra-segmento)

procedura _lung che _main sono NEAR

il passaggio di argomenti indirizzo assume sempre lo stesso segmento:

indirizzi intra-segmento

MODELLO SMALL

produzione eseguibile ===> **mains.exe**

tcc -ms main.c lungts.asm

Assembler corrispondente a main.c

===> esattamente uguale a prima

OSSERVAZIONI:

a) A TEMPO DI ESECUZIONE si ha un unico segmento per il codice ed uno per i dati (dati, stack, heap)

===> CS unico (_TEXT)

DS = SS (DGROUP -> _DATA, _BSS, _STACK)

il modello **cliente/servitore** usa sempre lo stesso segmento:

chiamate vicine (indirizzi intra-segmento)

il passaggio di argomenti indirizzo assume sempre lo stesso segmento:

indirizzi intra-segmento

b) sia la procedura _lung che _main sono NEAR

c) il chiamante (la funzione main) pone sullo STACK l'offset dell'array di caratteri corrispondente alla stringa

===> uso di puntatori NEAR

MODELLO COMPACT:

Programma ASSEMBLER (file lungc.asm):

```
_TEXT    segment byte public 'CODE'
public   _lung
;   int lung(char *s)
assume   cs:_TEXT
_lung proc      near
         push    bp
         mov     bp,sp
         push    si
;   {  int i=0;
         xor     si,si
;   while (*s++) i++;
ciclo:   les     bx,dword ptr [bp+4]
         inc     word ptr [bp+4]
         cmp     byte ptr es:[bx],0
         je      short fine
         inc     si
         jmp     ciclo
;   return i;
fine:    mov     ax,si
;   }
         pop     si
         pop     bp
         ret
_lung endp
_TEXT    ends
    end
```

MODELLO COMPACT

produzione eseguibile ===> **mainc.exe**

tcc -mc main.c lungc.asm

Assembler corrispondente a main.c ===> simile a prima

OSSERVAZIONI:

a) A TEMPO DI ESECUZIONE **unico segmento codice**
la procedura _lung che _main sono NEAR

b) A TEMPO DI ESECUZIONE si hanno **segmenti diversi**
per dati, stack ed heap
===> DS != SS != CS
DGROUP (_DATA, _BSS), _STACK
gli **indirizzi dei dati** sono specificati in modo far

Per le funzioni:
il cliente è sempre vicino al servitore
(modalità intra-segmento)
il cliente in caso di indirizzo nei parametri usa
modalità inter-segmento
pone sullo STACK segmento ed offset dell'array di caratteri corrispondente alla stringa
===> uso di puntatori FAR

MODELLO MEDIUM:

Programma ASSEMBLER (file lungm.asm):

```
LUNG_TEXT   segment byte public 'CODE'
public      _lung
;     int lung(char *s)
assume   cs: LUNG_TEXT
_lung proc       far
            push     bp
            mov      bp,sp
            push     si
;     { int i=0;
            xor      si,si
;     while (s[i]) i++;
ciclo:      mov      bx,word ptr [bp+6]
            inc      word ptr [bp+6]
            cmp      byte ptr [bx],0
            je       short fine
            inc      si
            jmp      ciclo
;     return i;
fine:       mov      ax,si
;     }
            pop      si
            pop      bp
            ret
_lung       endp
LUNG_TEXT   ends
      end
```

MODELLO MEDIUM

produzione eseguibile ===> **mainm.exe**

tcc -mm main.c lungm.asm

Assembler corrispondente a main.c ===>

```
DGROUP    group_DATA,_BSS
assume   cs: MAIN_TEXT, ds:DGROUP
MAIN_TEXT    segment byte public 'CODE'
;    void main()
assume   cs: MAIN_TEXT
_mainproc       far
          push      bp
          mov       bp,sp
          ...
          pop       bp
          ret
_mainendp
MAIN_TEXT    ends
_BSS  segment word public 'BSS'
_r        db        2 dup (?)
_BSS  ends
_DATA    segment word public 'DATA'
_string   db        'Ciao ciao!'
          db   0
          db   'Lunghezza stringa %d'
          ...
_DATA    ends
```


OSSERVAZIONI:

a) A TEMPO DI ESECUZIONE **segmenti codice diversi**
le procedure da un modulo ad un altro sono sempre FAR
_lung e _main sono FAR
segmenti **main_TEXT** e **lungm_TEXT**

b) A TEMPO DI ESECUZIONE si ha un **unico segmento dati** (statici, dinamici ed heap)
===> DS = SS
tutti gli **indirizzi dei dati** sono specificati in modo NEAR
segmento **DGROUP (_DATA, _BSS, altri)**

Per le funzioni:

il cliente è sempre near ad un servitore nello stesso file
il cliente è sempre far ad un servitore di un altro file
modalità inter-segmento

la call e ret salvano e ripristinano segmento ed offset per il ritorno al cliente
===> uso di indirizzi FAR

In caso di indirizzo nei parametri si usa solo offset
modalità intra-segmento

MODELLO LARGE:

Programma ASSEMBLER (file lungl.asm):

```
LUNG_TEXT    segment byte public 'CODE'
public     _lung
;     int lung(char *s)
assume    cs: LUNG_TEXT
_lung proc        far
            push      bp
            mov       bp,sp
            push      si
;     { int i=0;
            xor       si,si
;     while (*s++) i++;
ciclo:      les       bx,dword ptr [bp+6]
            inc       word ptr [bp+6]
            cmp       byte ptr es:[bx],0
            je        short fine
            inc       si
            jmp       ciclo
;     return i;
fine:       mov       ax,si
;     }
            pop       si
            pop       bp
            ret
_lung      endp
LUNG_TEXT    ends
     end
```


MODELLO LARGE

produzione eseguibile ===> **mainl.exe**

tcc -ml main.c lungl.asm

Assembler corrispondente a main.c ===> simile a prima

OSSERVAZIONI:

a) A TEMPO DI ESECUZIONE **segmenti codice diversi**
le procedure da un modulo ad un altro sono sempre FAR
segmenti **main_TEXT** e **lungl_TEXT**
_lung e _main sono FAR

b) A TEMPO DI ESECUZIONE **segmenti dati diversi**
(statici, dinamici ed heap)
===> DS != SS
tutti gli **indirizzi dei dati** sono specificati in modo FAR

il cliente è sempre near ad un servitore nello stesso file
il cliente è sempre far ad un servitore di un altro file
modalità inter-segmento (codice)
In caso di indirizzo nei parametri si usa indirizzo completo
modalità inter-segmento (dati)

MODELLO HUGE:

Programma ASSEMBLER (file lungh.asm):

```
LUNG_TEXT    segment byte public 'CODE'
public     _lung    ; int lung(char *s)
assume   cs: LUNG_TEXT, ds: LUNG_DATA
_lung proc       far
          push     bp
          mov      bp,sp
          push     si
          push     ds
          mov      ax, LUNG_DATA
          mov      ds,ax
          xor      si,si ;      { int i=0;
;    while (*s++) i++;
ciclo:    les      bx, dword ptr [bp+6]
          inc      word ptr [bp+6]
          cmp      byte ptr es:[bx],0
          je       short fine
          inc      si
          jmp      ciclo
fine:     mov      ax,si ;     return i;
;    }
          pop      ds
          pop      si
          pop      bp
          ret
_lung endp
LUNG_TEXT    ends
```



```
LUNG_DATA    segment word public 'DATA'
 ...
LUNG_DATA    ends
    end
```

MODELLO HUGE

produzione eseguibile ===> **mainh.exe**

tcc -mh main.c lungh.asm

Assembler corrispondente a main.c ===>

```
assume   cs:MAIN_TEXT, ds:MAIN_DATA
MAIN_TEXT    segment byte public 'CODE'
;    void main()
_main     proc     far
          push     bp
          mov      bp,sp
          push     ds
          mov      ax,MAIN_DATA
          mov      ds,ax
          ...
          pop      ds
          pop      bp
          ret
_main     endp
MAIN_TEXT    ends
```

```
MAIN_DATA    segment word public 'DATA'
; un solo segmento dati per quelli NON inizializzati
; ed inizializzati
_r          db          2 dup (0)
_string     db          'Ciao ciao!'
            db          0
            db          'Lunghezza stringa %d'
      ...
MAIN_DATA    ends
```

OSSERVAZIONI:

a) Per ogni modulo **segmenti codice diversi**
le procedure da un modulo ad un altro sono sempre FAR
_lung e _main sono FAR
segmenti **main_TEXT** e **lungh_TEXT**

b) Per ogni file un **segmento dati diverso**
===> **main_DATA, lungh_DATA**
tutti gli **indirizzi dei dati** sono specificati in modo FAR

il cliente è sempre near ad un servitore nello stesso file
il cliente è sempre far ad un servitore di un altro file
modalità inter-segmento
In caso di indirizzo nei parametri si usano indirizzi completi
modalità inter-segmento
===> uso di puntatori FAR

ESERCIZIO:
Dato il seguente frammento C convertire in Assembler per 8086 usando il modello SMALL

```
#define NELEM 20
static struct
{   int a[NELEM], *b; } dsc;
void fun2(void)
{  int j;
dsc.b = (int *) malloc(sizeof(int));
*dsc.b=0;
for (j = 0; j < NELEM; j++)
      *dsc.b += dsc.a[j];
}
```

Programma ASSEMBLER:

```
DGROUP      group     _DATA
_TEXT    segment byte public 'CODE'
            extrn      _malloc:near
            public     _fun2
;
;      void fun2(void)
            assume  cs:_TEXT,ds:DGROUP
_fun2       procnear
            push       bp
            mov        bp,sp
            push       si
;      {int j;
;       dsc.b = (int *) malloc(sizeof(int));
            mov        ax,2        ; sizeof(int)
            push       ax
```

```
            call    near ptr _malloc
            pop     cx
            mov     word ptr DGROUP:_dsc+40,ax
;       *dsc.b=0;
            mov     bx,word ptr DGROUP:_dsc+40
            mov     word ptr [bx],0
;       for (j = 0; j < NELEM; j++)
            xor     si,si
ciclo:      cmp     si,20
            jge     short fine
;           *dsc.b += dsc.a[j];
            mov     bx,si
            shl     bx,1
            mov     ax,word ptr DGROUP:_dsc[bx]
            mov     bx,word ptr DGROUP:_dsc+40
            add     word ptr [bx],ax
            inc     si
            jmp     ciclo
fine:       pop si
            pop bp
            ret
_fun2       endp
_TEXT       ends
_DATA       segment word public 'DATA'
_dsc        db      40 dup (?)
            dw      ?
_DATA       ends
            end
```


ESERCIZIO 2:

Dato il seguente frammento C convertire in Assembler per 8086 usando il modello SMALL

```
fun2(int x[], int x1, int x2)
{ int j, *y = x;
for (j = 1; *y; j++, y++) if (*y < x1 || x2 < *y) return j;
if (j == 1) return 0; else return fun2(x, x1+1, x2 -1);
}
_TEXT   segment byte public 'CODE'
;       fun2(int x[], int x1, int x2)
            public      _fun2
            assume      cs:_TEXT
_fun2       procnear
            push        bp
            mov         bp,sp
            push        si
            push        di
;       { int j, *y = x;
            mov         si,word ptr [bp+4]
;        for (j = 1; *y; j++, y++)
            mov         di,1
ciclo:      cmp         word ptr [si],0
            je          short finec
;           if (*y < x1 || x2 < *y) return j;
            mov         ax,word ptr [si]
            cmp         ax,word ptr [bp+6]
            jl          short esci
            mov         ax,word ptr [si]
            cmp         ax,word ptr [bp+8]
            jle         short cont
```

```
esci:   mov     ax,di
        jmp     short fine
cont:   inc     di
        add     si, 2
        jmp ciclo
    ;    if (j == 1) return 0;
finec:  cmp     di,1
        jne     short vai
        xor     ax,ax
        jmp     short fine
;   else return fun2(x, x1+1, x2 -1);
vai:    mov     ax,word ptr [bp+8]
        dec     ax
        push    ax
        mov     ax,word ptr [bp+6]
        inc     ax
        push    ax
        push    word ptr [bp+4]
        call    near ptr _fun2
        add     sp,6
    ;   }
fine:   pop     di
        pop     si
        pop     bp
        ret
_fun2   endp
_TEXT   ends
        end
```